

PIPE FAILURE PREDICTION(KNN MODEL)

September 11, 2026

```
[2]: import pandas as pd
data = pd.read_csv(r"C:\Users\User\Downloads\DATA ANALYSIS PORTFOLIO\5 WATER_
↳PROJECTS\04_pipe_failure_prediction.csv")
data.head()
```

```
[2]:
```

	pipe_id	diameter_mm	material	installation_year	age_years	length_m	\
0	P-00001	63	PVC	2016	10	1328.6	
1	P-00002	50	GI	2011	15	690.1	
2	P-00003	63	HDPE	2013	13	2073.1	
3	P-00004	160	PVC	2012	14	1867.8	
4	P-00005	40	PVC	2022	4	2119.9	

	operating_pressure_m	soil_type	elevation_m	previous_failures	\
0	39.0	Clay	1478.0	0	
1	11.1	Rocky	1768.3	0	
2	34.2	Sandy	809.2	1	
3	44.8	Clay	948.3	2	
4	31.8	Black Cotton	1012.5	0	

	leakage_incidents	annual_rainfall_mm	recent_rainfall_mm	\
0	3	672.4	0.6	
1	1	956.3	6.9	
2	1	1331.9	7.6	
3	1	1323.1	8.7	
4	0	882.4	3.2	

	maintenance_count_24m	failure_within_90_days	failure_risk_score	\
0	4	1	12.7	
1	3	0	8.1	
2	3	0	14.4	
3	3	0	24.2	
4	2	0	7.5	

	risk_category
0	Low
1	Low
2	Low

```
3          Low
4          Low
```

```
[3]: data.shape
      #6000 rows(observations) and 17 columns/variables
```

```
[3]: (6000, 17)
```

```
[4]: data.columns
```

```
[4]: Index(['pipe_id', 'diameter_mm', 'material', 'installation_year', 'age_years',
          'length_m', 'operating_pressure_m', 'soil_type', 'elevation_m',
          'previous_failures', 'leakage_incidents', 'annual_rainfall_mm',
          'recent_rainfall_mm', 'maintenance_count_24m', 'failure_within_90_days',
          'failure_risk_score', 'risk_category'],
          dtype='object')
```

```
[6]: data.info()
      #there are no missing values in any of the columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   pipe_id                               6000 non-null   object
1   diameter_mm                           6000 non-null   int64
2   material                               6000 non-null   object
3   installation_year                      6000 non-null   int64
4   age_years                             6000 non-null   int64
5   length_m                              6000 non-null   float64
6   operating_pressure_m                  6000 non-null   float64
7   soil_type                             6000 non-null   object
8   elevation_m                           6000 non-null   float64
9   previous_failures                     6000 non-null   int64
10  leakage_incidents                     6000 non-null   int64
11  annual_rainfall_mm                    6000 non-null   float64
12  recent_rainfall_mm                    6000 non-null   float64
13  maintenance_count_24m                 6000 non-null   int64
14  failure_within_90_days                 6000 non-null   int64
15  failure_risk_score                     6000 non-null   float64
16  risk_category                         6000 non-null   object
dtypes: float64(6), int64(7), object(4)
memory usage: 797.0+ KB
```

```
[7]: data.isnull().sum()
```

```
[7]: pipe_id          0
      diameter_mm     0
      material        0
      installation_year 0
      age_years       0
      length_m        0
      operating_pressure_m 0
      soil_type       0
      elevation_m     0
      previous_failures 0
      leakage_incidents 0
      annual_rainfall_mm 0
      recent_rainfall_mm 0
      maintenance_count_24m 0
      failure_within_90_days 0
      failure_risk_score 0
      risk_category   0
      dtype: int64
```

```
[8]: #EXAMINE THE TARGET. Our target is failure_within_90_days
      data['failure_within_90_days'].value_counts()

#Rows with observation(0) are 4426 and rows with observation(1) are 1574
```

```
[8]: failure_within_90_days
      0    4426
      1    1574
      Name: count, dtype: int64
```

```
[9]: data['failure_within_90_days'].value_counts(normalize=True)*100

#What proportion of the pipes actually fail within 90 days? 26.23%
```

```
[9]: failure_within_90_days
      0    73.766667
      1    26.233333
      Name: proportion, dtype: float64
```

```
[10]: data.groupby('failure_within_90_days').mean(numeric_only=True)

#this gives us an initial engineering indication of what may be associated with failure
↪ failure
#failure tends to happen in pipes with a diameter of 69.844mm
#failure happens in pipes installed years ago(2004.95) (older pipes)
#failure happens in much older pipes(21.04 years)
#it happens in quite shorter pipe lengths(1267.88m)
#it happens when operating pressure is higher(38.11m)
```

```
#it happens in pipes with a higher leakage incidents(2.23)
#it happens in pipelines with a higher maintenance count(4)
#risk failure score is high(38.75)
```

```
[10]:
      diameter_mm  installation_year  age_years  \
failure_within_90_days
0              71.000452          2011.299141  14.700859
1              69.844346          2004.958069  21.041931

      length_m  operating_pressure_m  elevation_m  \
failure_within_90_days
0          1285.306801              34.116810  1289.726661
1          1267.885642              38.110483  1285.286277

      previous_failures  leakage_incidents  \
failure_within_90_days
0              1.753050              1.411207
1              2.881194              2.235070

      annual_rainfall_mm  recent_rainfall_mm  \
failure_within_90_days
0          1050.335811              4.500926
1          1044.268424              4.692313

      maintenance_count_24m  failure_risk_score
failure_within_90_days
0              2.975373              22.524559
1              4.017154              38.749365
```

```
[18]: #IDENTIFY/GET OUR FEATURES(X) AND TARGET(Y)
#our features are all the columns except failure_within_90_days and pipe_id(it
↳does not tell KNN anything meaningful)
#our target is failure_within_90_days
x = data.drop(['failure_within_90_days', 'pipe_id', "failure_risk_score",
↳"risk_category"], axis=1)
y = data['failure_within_90_days']

# failure_risk_score and risk_category sound like pre-existing risk assessments
↳derived from the other pipe variables. If they were calculated using the
↳same failure outcome we're trying to predict, using them would create data
↳leakage.
```

```
[19]: print(x.shape)
print(y.shape)

#x features have 6000 observations and 13 columns
#y features have 6000 observations and 1 column
```

```
(6000, 13)
(6000,)
```

```
[20]: x.columns
```

```
[20]: Index(['diameter_mm', 'material', 'installation_year', 'age_years', 'length_m',
        'operating_pressure_m', 'soil_type', 'elevation_m', 'previous_failures',
        'leakage_incidents', 'annual_rainfall_mm', 'recent_rainfall_mm',
        'maintenance_count_24m'],
        dtype='object')
```

```
[21]: x.head()
```

```
[21]:
```

	diameter_mm	material	installation_year	age_years	length_m	\
0	63	PVC	2016	10	1328.6	
1	50	GI	2011	15	690.1	
2	63	HDPE	2013	13	2073.1	
3	160	PVC	2012	14	1867.8	
4	40	PVC	2022	4	2119.9	

	operating_pressure_m	soil_type	elevation_m	previous_failures	\
0	39.0	Clay	1478.0	0	
1	11.1	Rocky	1768.3	0	
2	34.2	Sandy	809.2	1	
3	44.8	Clay	948.3	2	
4	31.8	Black Cotton	1012.5	0	

	leakage_incidents	annual_rainfall_mm	recent_rainfall_mm	\
0	3	672.4	0.6	
1	1	956.3	6.9	
2	1	1331.9	7.6	
3	1	1323.1	8.7	
4	0	882.4	3.2	

	maintenance_count_24m
0	4
1	3
2	3
3	3
4	2

```
[22]: #IDENTIFY CATEGORICAL VARIABLES
x.select_dtypes(include='object').columns

#these are columns containing text/categories
#these can not be directly fed into KNN or logistic regression
#a model cannot calculate a meaningful mathematical distance between pvc and
↳hdpe
```

```
# we will therefore encode them
```

```
[22]: Index(['material', 'soil_type'], dtype='object')
```

```
[24]: #IDENTIFY NUMERICAL VARIABLES/COLUMNS
x.select_dtypes(include=['int64', 'float64']).columns

#numerical columns can be fed into KNN and Logistic Regression and be used
↳numerically
```

```
[24]: Index(['diameter_mm', 'installation_year', 'age_years', 'length_m',
            'operating_pressure_m', 'elevation_m', 'previous_failures',
            'leakage_incidents', 'annual_rainfall_mm', 'recent_rainfall_mm',
            'maintenance_count_24m'],
            dtype='object')
```

```
[27]: #SPLIT THE DATA INTO TRAINING AND TESTING SETS BEFORE SCALING
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 20% of the data set will be used for testing
# 80% of the dataset will be used for training#the model learns from 4800
↳training observations
# we test it on 1200 observations it has never seen
# Why stratify=y?This is particularly important if failures and non-failures
↳aren't evenly distributed.
# It makes sure the training and testing sets retain approximately the same
↳proportion of both
# Our dataset has some potentially important columns like: failure_risk_score
↳and risk_category
# We need to determine whether these are legitimate predictors or whether they
↳were calculated using information related to failure_within_90_days.
# If they're derived from the failure outcome, including them would be data
↳leakage and would make our model unrealistically good.
```

```
[28]: print('Training set:', x_train.shape)
print('Testing set:', x_test.shape)
```

Training set: (4800, 13)

Testing set: (1200, 13)

```
[30]: #SCALING/STANDARDIZING THE NUMERICAL COLUMNS
#the numerical columns have values that are of very different ranges,
#without scaling, a variable with large numerical values such as length_m could
↳dominate the distance calculation

from sklearn.preprocessing import StandardScaler

#DEFINE THE NUMERICAL COLUMNS
numerical_features = x.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

print(numerical_features)
```

```
['diameter_mm', 'installation_year', 'age_years', 'length_m',
'operating_pressure_m', 'elevation_m', 'previous_failures', 'leakage_incidents',
'annual_rainfall_mm', 'recent_rainfall_mm', 'maintenance_count_24m']
```

```
[31]: scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train[numerical_features])
```

```
[32]: x_test_scaled = scaler.transform(x_test[numerical_features])
```

```
[34]: #ENCODING CATEGORICAL VARIABLES/COLUMNS
from sklearn.preprocessing import OneHotEncoder

categorical_features = x.select_dtypes(
    include="object"
).columns.tolist()

print(categorical_features)
```

```
['material', 'soil_type']
```

```
[35]: #CREATE THE ENCODER
encoder = OneHotEncoder(handle_unknown='ignore')
```

```
[38]: #ENCODE THE CATEGORICAL TESTING AND TRAINING DATA
x_train_enc = encoder.fit_transform(x_train[categorical_features])
```

```
[39]: x_test_enc = encoder.transform(x_test[categorical_features])

# What have we created? At this point we have four objects:
# x_train_scaled, x_test_scaled

# x_train_enc, x_test_enc
# The numerical variables have been standardized, while the categorical
↳variables have been converted into numerical dummy variables.
```

```
[40]: # However, there's one issue: StandardScaler gives us a normal NumPy array,
      # ↳ while OneHotEncoder normally gives us a sparse matrix.
      # So before giving everything to KNN, we'll combine them.
      from scipy.sparse import hstack

      x_train_processed = hstack([
          x_train_scaled,
          x_train_enc
      ])

      x_test_processed = hstack([
          x_test_scaled,
          x_test_enc
      ])

      #      X_train_processed
      #      ↓
      #      Scaled numerical variables
      #      +
      #      Encoded categorical variables
      #      ↓
      #      Ready for KNN
```

```
[41]: #BUILD THE KNN MODEL
      from sklearn.neighbors import KNeighborsClassifier
      knn = KNeighborsClassifier(n_neighbors = 5)

      #When predicting whether a pipe will fail, look at the 5 most similar pipes in
      #↳ the training data and use their outcomes to make the prediction.
```

```
[42]: #TRAIN THE MODEL
      # This is where KNN learns from your training data.
      # Unlike some algorithms, KNN doesn't create a complicated mathematical
      # ↳ equation during training. It essentially
      # retains the training observations and uses their distances when making
      # ↳ predictions.
      knn.fit(x_train_processed, y_train)
```

```
[42]: KNeighborsClassifier()
```

```
[43]: #MAKE PREDICTIONS
      #Now give the model the unseen testing data:
      y_pred_knn = knn.predict(x_test_processed)
```

```
[44]: # Let's look at the first 20 predictions:
      y_pred_knn[:20]
```



```
# 0 = No failure within 90 days(18 observations)
# 1 = Failure within 90 days (2 observations)
```

```
[44]: array([0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
[45]: #EVALUATE THE MODEL
#First, let's calculate accuracy:
from sklearn.metrics import accuracy_score

accuracy_knn = accuracy_score(y_test, y_pred_knn)

print("KNN Accuracy:", accuracy_knn)
```

```
KNN Accuracy: 0.7316666666666667
```

```
[46]: # CLASSIFICATION REPORT
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_knn))

# Pay particular attention to class 1. That is our Pipe failure within 90 days
# PRECISION: Of the pipes KNN predicted would fail: How many actually failed? 0.
↪48
# RECALL: Of all the pipes that actually failed: How many did KNN successfully
↪identify? 0.27
# t means the model identified approximately 27% of the pipes that actually
↪failed.
```

	precision	recall	f1-score	support
0	0.78	0.90	0.83	885
1	0.48	0.27	0.35	315
accuracy			0.73	1200
macro avg	0.63	0.58	0.59	1200
weighted avg	0.70	0.73	0.70	1200

```
[47]: #CONFUSION MATRIX
from sklearn.metrics import confusion_matrix

cm_knn = confusion_matrix(y_test, y_pred_knn)

print(cm_knn)
```

```
[[793  92]
 [230  85]]
```

```
[51]: # TEST DIFFERENT K VALUES
```

```
accuracy_scores = []
```

```
for k in range(1, 21):
```

```
    knn_model = KNeighborsClassifier(n_neighbors=k)
```

```
    knn_model.fit(x_train_processed, y_train)
```

```
    y_pred = knn_model.predict(x_test_processed)
```

```
    accuracy = accuracy_score(y_test, y_pred)
```

```
    accuracy_scores.append(accuracy)
```

```
[52]: for k, accuracy in zip(range(1, 21), accuracy_scores):
```

```
    print(f"K = {k}: Accuracy = {accuracy:.4f}")
```

```
K = 1: Accuracy = 0.6475
```

```
K = 2: Accuracy = 0.7225
```

```
K = 3: Accuracy = 0.7200
```

```
K = 4: Accuracy = 0.7333
```

```
K = 5: Accuracy = 0.7317
```

```
K = 6: Accuracy = 0.7417
```

```
K = 7: Accuracy = 0.7233
```

```
K = 8: Accuracy = 0.7350
```

```
K = 9: Accuracy = 0.7300
```

```
K = 10: Accuracy = 0.7408
```

```
K = 11: Accuracy = 0.7383
```

```
K = 12: Accuracy = 0.7425
```

```
K = 13: Accuracy = 0.7392
```

```
K = 14: Accuracy = 0.7408
```

```
K = 15: Accuracy = 0.7408
```

```
K = 16: Accuracy = 0.7392
```

```
K = 17: Accuracy = 0.7367
```

```
K = 18: Accuracy = 0.7383
```

```
K = 19: Accuracy = 0.7367
```

```
K = 20: Accuracy = 0.7450
```

```
[ ]: # Your best accuracy among K=1-20 is: K = 20 → 74.50%
```

```
# The next best are:
```

```
# K = 12 → 74.25%
```

```
# K = 6 → 74.17%
```

```
# K = 10 → 74.08%
```

```
# K = 14/15 → 74.08%
```

```
# So K=20 is currently the best by accuracy in your test. However, I don't want  
→ us to declare K=20 the final K yet.
```

```

# We selected K by looking directly at the test set. That's not ideal because
↳ the test set is supposed to represent our final unbiased evaluation.
# Instead, we'll use cross-validation on the training data to choose K.
# The process becomes:
#
#           Training data (4,800)
#           ↓
#           Cross-validation
#           ↓
#           Choose best K
#           ↓
#           Train final KNN
#           ↓
#           Test ONCE on 1,200 unseen pipes
#           ↓
#           Final performance

```

```

[53]: # Find the best K using cross-validation
from sklearn.model_selection import cross_val_score

cv_scores = []

for k in range(1, 21):

    knn_model = KNeighborsClassifier(n_neighbors=k)

    scores = cross_val_score(
        knn_model,
        x_train_processed,
        y_train,
        cv=5,
        scoring="accuracy"
    )

    cv_scores.append(scores.mean())

# What does cv=5 mean? We're taking your 4,800 training pipes and dividing them
↳ into 5 parts.
# Fold 1 → Train on 4 parts, validate on 1
# Fold 2 → Train on 4 parts, validate on another
# Fold 3 → ...
# Fold 4 → ...
# Fold 5 → ...

```

```

[54]: # display the results
for k, score in zip(range(1, 21), cv_scores):
    print(f"K = {k}: CV Accuracy = {score:.4f}")

```

```

K = 1: CV Accuracy = 0.6775
K = 2: CV Accuracy = 0.7294
K = 3: CV Accuracy = 0.7144
K = 4: CV Accuracy = 0.7344
K = 5: CV Accuracy = 0.7327
K = 6: CV Accuracy = 0.7417
K = 7: CV Accuracy = 0.7400
K = 8: CV Accuracy = 0.7402
K = 9: CV Accuracy = 0.7421
K = 10: CV Accuracy = 0.7433
K = 11: CV Accuracy = 0.7423
K = 12: CV Accuracy = 0.7425
K = 13: CV Accuracy = 0.7465
K = 14: CV Accuracy = 0.7429
K = 15: CV Accuracy = 0.7442
K = 16: CV Accuracy = 0.7435
K = 17: CV Accuracy = 0.7444
K = 18: CV Accuracy = 0.7448
K = 19: CV Accuracy = 0.7412
K = 20: CV Accuracy = 0.7425

```

```

[55]: # Then find the best K automatically:
      best_k = range(1, 21)[cv_scores.index(max(cv_scores))]

      print("Best K:", best_k)
      print("Best CV Accuracy:", max(cv_scores))

```

```

Best K: 13
Best CV Accuracy: 0.7464583333333332

```

```

[56]: #Train the final KNN
      final_knn = KNeighborsClassifier(n_neighbors=best_k)

      final_knn.fit(x_train_processed, y_train)

```

```

[56]: KNeighborsClassifier(n_neighbors=13)

```

```

[57]: y_pred_knn = final_knn.predict(x_test_processed)

```

```

[58]: # Evaluate the final KNN
      from sklearn.metrics import accuracy_score

      knn_accuracy = accuracy_score(y_test, y_pred_knn)

      print(f"KNN Test Accuracy: {knn_accuracy:.4f}")

```

```

KNN Test Accuracy: 0.7392

```

```
[59]: from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_knn))
```

	precision	recall	f1-score	support
0	0.77	0.92	0.84	885
1	0.51	0.24	0.32	315
accuracy			0.74	1200
macro avg	0.64	0.58	0.58	1200
weighted avg	0.70	0.74	0.70	1200

```
[60]: from sklearn.metrics import confusion_matrix

knn_cm = confusion_matrix(y_test, y_pred_knn)

print(knn_cm)
```

```
[[812  73]
 [240  75]]
```

```
[ ]:
```